SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

Course Code:	CSA10 / CSA1021	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 2 – **Agile Sprint Planning & User Story Workshop**
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	Bhuvaneshwaran J	**Reg. No.:**	192511402
Date:	15/10/2026	**Duration:**	60 minutes

Code of Conduct

I Bhuvaneshwaran J(192511402) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Bhuvaneshwaran J

Annexure A – Agile Sprint Planning & User Story Workshop

Instructions to Students:

Each student is assigned an individual software project problem statement. Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

1. **Identify the Project Vision and Stakeholders**
   - Define the project's vision, objectives, and identify all key stakeholders involved.
2. **Create Epics and User Stories**
   - Break down the requirements into Epics and formulate well-structured User Stories using the format: *As a <user>, I want <feature>, so that <benefit>*.
3. **Define Acceptance Criteria**
   - Write clear and measurable acceptance criteria for each selected user story using the Given–When–Then format or equivalent.
4. **Prioritize the Product Backlog**
   - Organize and prioritize the product backlog based on business value, customer needs, and project dependencies.
5. **Plan the Sprint Backlog**
   - Select high-priority user stories for the first sprint and divide them into actionable development tasks.
6. **Estimate Effort Using Story Points**
   - Assign story points to each selected user story using an appropriate estimation technique (e.g., Fibonacci sequence or Planning Poker) and justify your estimates.
7. **Present the Sprint Goal and Expected Deliverables**
   - Define a clear sprint goal and describe the expected sprint deliverables, including the features to be completed at the end of the sprint.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification (30%)	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.
Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	Mark
Project Vision & Stakeholder Identification **(30%)**	/5
Epics & User Stories **(25%)**	/5
Acceptance Criteria **(20%)**	/5
Product Backlog Prioritization & Sprint Planning **(15%)**	/5
Story Point Estimation, Sprint Goal & Presentation **(10%)**	/5
TOTAL	/25

Industry Context:

International banking transfers via legacy networks like SWIFT remain slow, expensive, and non-transparent, taking days to settle across different time zones, clearinghouses, and currency pairings. These delays trap billions of dollars in dormant capital within intermediary banks to cover settlement risks. A real-time, cross-border transactional architecture must bypass regional silos while maintaining precise compliance, liquidity management, and foreign exchange execution.

Problem Statement:

Design and develop a distributed, high-availability Cross-Border Settlement & Liquidity Routing Protocol that executes multi-currency financial transactions instantly. The system must monitor real-time treasury balances, foreign exchange volatility, regional compliance blacklists, and transaction fees across global liquidity hubs. It must dynamically determine the most capital-efficient path to route, convert, and settle a transaction across international borders within seconds.

Objectives:

Coordinate multi-currency transaction state transitions with zero-error atomic consistency.

Optimize transaction routing paths across global banking nodes using dynamic graph weights.

Integrate automated Sanction Screening and AML checks within the transaction lifecycle.

Maintain highly accurate, synchronized ledgers across geographically distributed nodes.

Formulate fallback strategies for instant transaction reversal during node connectivity drops.

Expected Outcomes:

Reduction of cross-border settlement overhead times from days to sub-second windows.

Minimized foreign exchange exposure risks for multinational enterprise transactions.

Elimination of manual reconciliation efforts between intermediary banking

systems.
Lower operational fees passed down to international remittance senders.

1.1 Project vision

To develop a distributed, highly available cross-border settlement and liquidity routing protocol that evaluates real-time treasury balances, foreign-exchange conditions, regional sanctions and AML restrictions, transaction fees, and banking-node availability to select a capital-efficient route for multi-currency transfers. The system aims to reduce avoidable settlement delays and costs while preserving compliance, transaction consistency, security, and traceability. Sub-second processing is a target for eligible transaction paths, subject to external banking networks, liquidity, and regulatory constraints.

The vision can be expressed in one sentence for the Scrum team: **"Every cross-border transfer is validated, compliant, routed through the most efficient eligible path, settled exactly once, and fully traceable."**

1.2 Project objectives

- Coordinate multi-currency transaction state transitions with atomic consistency and prevent duplicate or partial processing.
- Optimize routes across global banking nodes using dynamic graph weights based on cost, liquidity, FX conditions, time, and risk.
- Integrate sanctions screening and Anti-Money Laundering (AML) checks into the transaction lifecycle.
- Maintain accurate, synchronized transaction and settlement records across geographically distributed nodes.
- Provide safe fallback, retry, and compensation/reversal procedures when nodes or network connections fail.
- Reduce settlement overhead, foreign-exchange exposure, manual reconciliation, and avoidable transfer fees.

1.3 Key stakeholders and responsibilities

Table 3: Stakeholders, roles, and expectations

Stakeholder	Responsibilities	Main expectation
Sending bank / originating institution	Submits validated transfer instructions, verifies the sender, and confirms funding.	Fast, accurate acceptance and clear rejection reasons.
Receiving bank / beneficiary institution	Receives settlement instructions, credits the beneficiary when appropriate, and confirms completion.	Reliable instructions and timely confirmation.
Intermediary banks and global liquidity hubs	Provide corridor access, liquidity, settlement services, balances, fees, and node-status information.	Accurate routing volumes and predictable settlement.
Treasury and liquidity managers	Monitor currency balances, funding needs, liquidity limits, and exposure.	Efficient use of capital and early visibility of shortfalls.
FX providers / market-data services	Supply exchange rates, spreads, quote expiry, and volatility data.	Quotes are used within their validity period.
Compliance and AML specialists	Maintain screening rules, review alerts, and decide on escalated or suspicious transactions.	No prohibited transaction settles; manageable alert volume.
Corporate and remittance customers	Initiate transfers, provide correct beneficiary details, and require transparent cost and status information.	Low cost, speed, and visible status.

Stakeholder	Responsibilities	Main expectation
Product owner	Defines business priorities, maintains the product backlog, and accepts completed features.	Maximum business value per sprint.
Scrum master	Facilitates Scrum events, removes impediments, and supports the team in following Agile practices.	A self-organizing, unblocked team.
Development, QA, security, and operations teams	Design, implement, test, secure, deploy, monitor, and maintain the protocol.	Clear requirements and sustainable pace.
Regulators and auditors	Define applicable obligations and inspect compliance, transaction evidence, and audit trails.	Complete, tamper-evident evidence.

1.4 Stakeholder interaction map

Figure 1 shows how the protocol sits between the payment participants, the financial-data providers, the governance bodies, and the delivery team. The arrows are two-way because each party both supplies information to the protocol and relies on information from it.

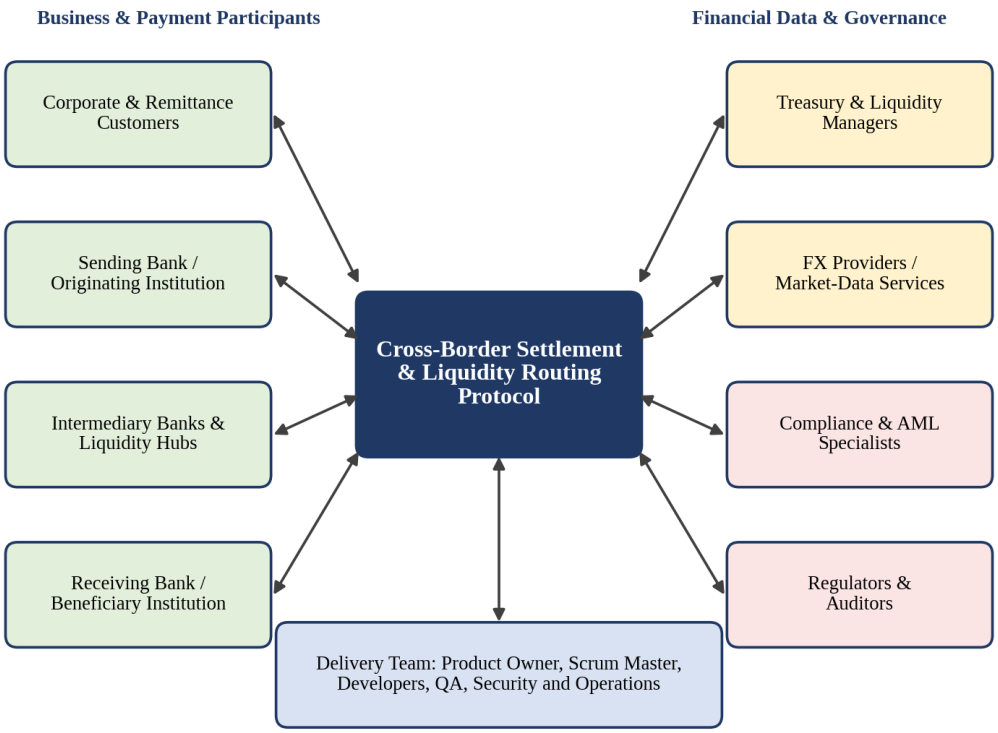


Figure 1: Stakeholder interaction map of the settlement protocol

1.5 Stakeholder influence and engagement

Stakeholders are grouped by their influence over the project and their interest in the outcome. The engagement approach follows from the grouping and is reviewed at each Sprint Review.

Table 4: Stakeholder engagement plan

Group	Influence	Interest	Engagement approach
Compliance and AML specialists	High	High	Attend refinement for US-03; approve screening outcomes and sandbox rules.

Group	Influence	Interest	Engagement approach
Regulators and auditors	High	Medium	Consulted on audit trail design; receive evidence packs before pilot.
Treasury and liquidity managers	High	High	Review liquidity rules and buffers; attend Sprint Reviews.
Sending and receiving banks	High	High	Joint integration testing in sandbox; interface agreements.
FX providers and intermediary banks	Medium	Medium	Interface specification and service-level agreements.
Corporate and remittance customers	Medium	High	Feedback on status messages and cost transparency.
Delivery team	High	High	Daily collaboration; all Scrum events.

1.6 Scope, assumptions, and constraints

In scope: transaction intake and validation, sanctions and AML screening, liquidity and FX visibility, dynamic route selection, settlement-state coordination, failure recovery, audit trail, and reconciliation. **Out of scope for the first release:** retail mobile applications, customer onboarding (KYC), cryptocurrency or stablecoin rails, and predictive liquidity analytics (kept as a future P3 item).

Assumptions:

- Partner banks and data providers can supply sandbox or test interfaces during the early sprints.
- Sprint 1 uses simulated or sandbox integrations and does not move real funds.
- Applicable regulatory obligations are provided by the compliance team before screening rules are finalized.
- The same Scrum team stays together so that velocity can be measured and used for forecasting.

Constraints:

- Sub-second processing depends on external networks and cannot be guaranteed for the full end-to-end settlement.
- Financial data and personal data must be protected; logs must not expose sensitive information.
- Compliance controls must never be bypassed to improve speed.

2. Create Epics and User Stories

User stories are written in the required format: “As a [user], I want [feature], so that [benefit].” Story IDs connect each story to its acceptance criteria, backlog position, and sprint tasks. Epics are large bodies of work that group related stories; each epic delivers one capability that stakeholders can recognize.

2.1 Epic and story hierarchy

Figure 2 shows how the product vision is decomposed into six epics and ten stories. Stories shaded green are selected for Sprint 1; the others are planned for later sprints.

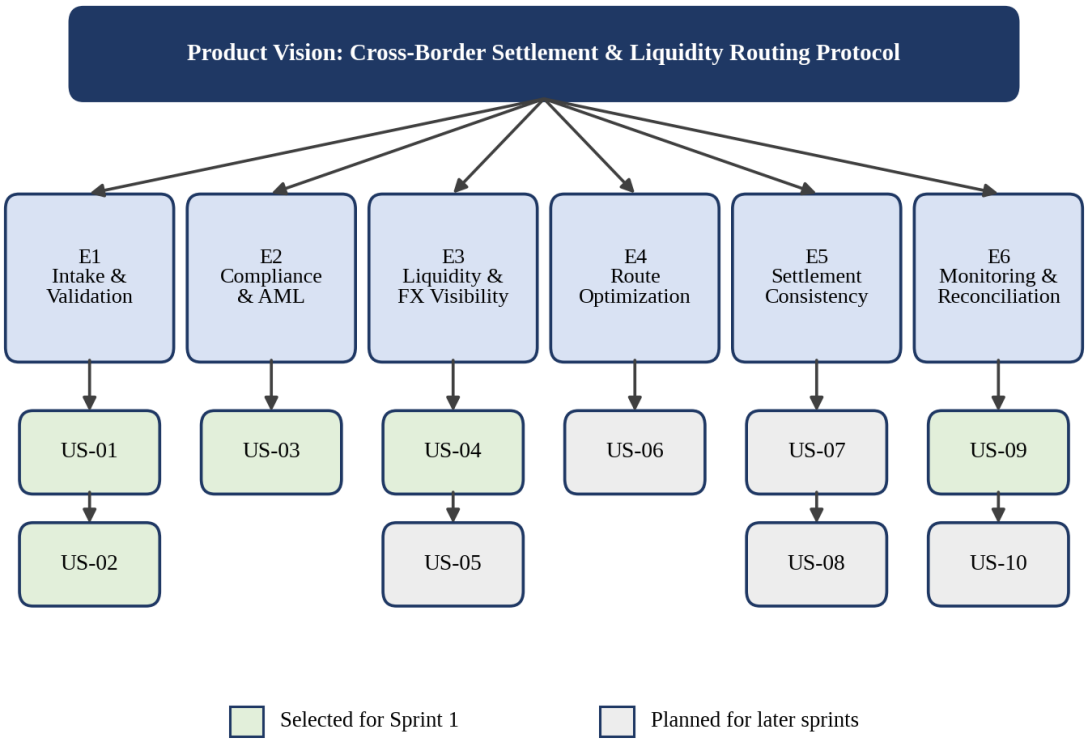


Figure 2: Product vision, epics, and user stories

2.2 Epics and user stories

Epic E1 – Transaction Intake and Validation

- **US-01:** As a sending-bank operator, I want transfer instructions validated for required fields, amount, currency, and beneficiary details, so that invalid transactions are rejected before processing.
- **US-02:** As a transaction service, I want every transfer to receive a unique transaction ID and idempotency key, so that duplicate requests do not create duplicate payments.

Epic E2 – Compliance and AML

- **US-03:** As a compliance officer, I want each transaction screened against applicable sanctions and AML rules before routing, so that restricted or suspicious transactions are blocked or escalated.

Epic E3 – Liquidity and FX Visibility

- **US-04:** As a treasury manager, I want current available balances and liquidity limits from eligible nodes, so that routes are not selected without sufficient funds.
- **US-05:** As a treasury or FX operator, I want valid FX quotes, spreads, expiry times, and volatility indicators, so that currency conversion is evaluated using current market information.

Epic E4 – Dynamic Route Optimization

- **US-06:** As a payment-routing service, I want to rank eligible routes by fees, expected time, liquidity, FX cost, and risk, so that the system can select a capital-efficient path.

Epic E5 – Settlement Consistency and Recovery

- **US-07:** As a settlement coordinator, I want explicit transaction states and participant confirmations, so that settlement status remains consistent and auditable.
- **US-08:** As an operations engineer, I want node-failure detection and safe fallback or compensation procedures, so that failed routes can be recovered without duplicate settlement.

Epic E6 – Monitoring and Reconciliation

- **US-09:** As an operations analyst, I want transaction status, route decisions, fees, and exceptions recorded in an audit trail, so that transfers can be tracked and investigated.
- **US-10:** As a reconciliation analyst, I want records from participating nodes compared automatically, so that mismatches are detected and manual reconciliation is reduced.

2.3 Quality check of the stories (INVEST)

Each story was reviewed against the INVEST criteria (Independent, Negotiable, Valuable, Estimable, Small, Testable). Stories that failed a criterion were split or reworded during the workshop. The summary below records the main finding for each story.

Table 5: INVEST review of user stories

Story	I	N	V	E / S	T	Comment
US-01	Yes	Yes	Yes	Yes	Yes	Independent entry point; easy to test.
US-02	Yes	Yes	Yes	Yes	Yes	Needs only validated input from US-01.
US-03	Partly	Yes	Yes	Yes	Yes	Needs a sandbox screening interface.
US-04	Yes	Yes	Yes	Yes	Yes	Mock provider allows early delivery.
US-05	Yes	Yes	Yes	Yes	Yes	Quote expiry logic is the main risk.
US-06	Partly	Yes	Yes	Large	Yes	Needs US-03, US-04, US-05; may be split.
US-07	Partly	Yes	Yes	Large	Yes	Split by state group if needed.
US-08	Partly	Yes	Yes	Large	Yes	Needs US-07; delivered in two parts.
US-09	Yes	Yes	Yes	Yes	Yes	Shares the event model with US-07.
US-10	Partly	Yes	Yes	Large	Yes	Needs stable events; later sprint.

2.4 End-to-end transaction workflow

Figure 3 shows the lifecycle that the ten stories implement. A transfer enters at step 1 and passes through validation, idempotency control, screening, liquidity and FX evaluation, routing, settlement, and audit. Red boxes show the controlled exits: a transaction that fails a check is stopped, queued, or recovered instead of continuing along an unsafe path.

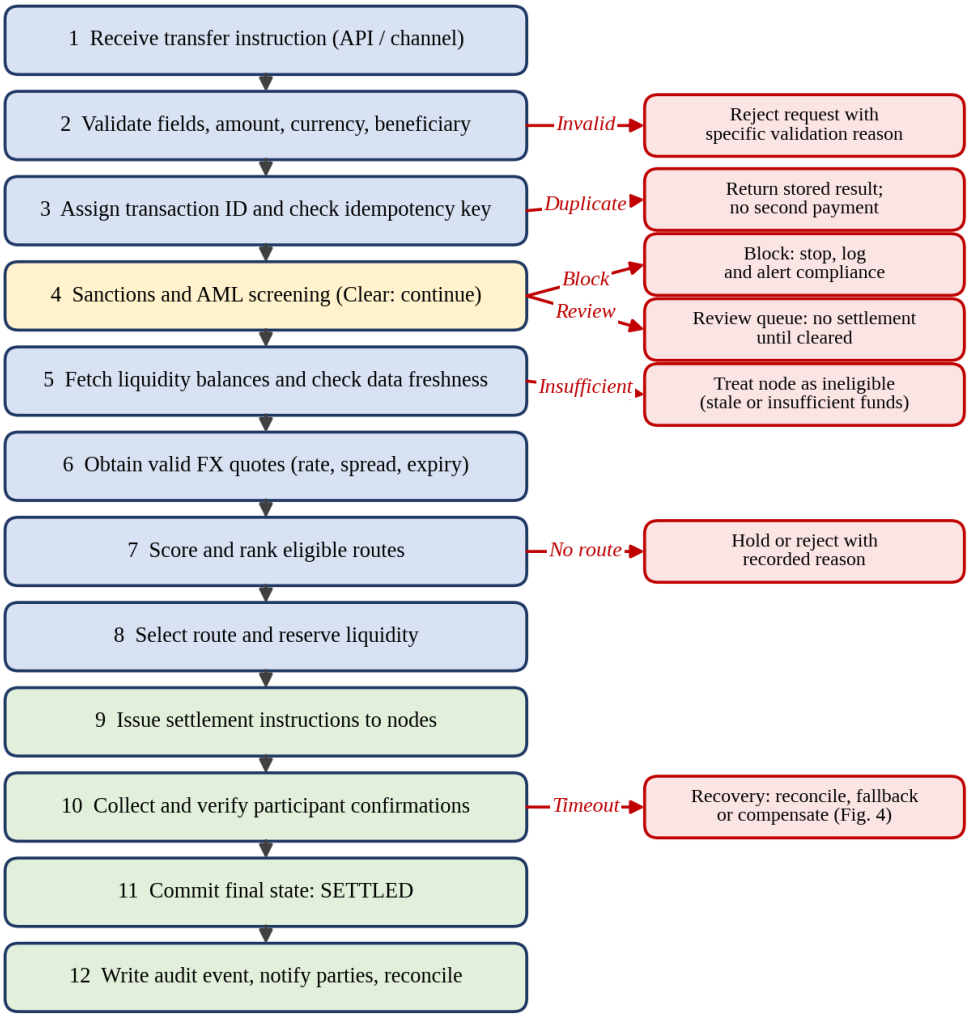


Figure 3: End-to-end transaction processing workflow

The workflow follows three design rules that appear throughout the acceptance criteria: (1) compliance screening always happens before routing and settlement; (2) a route is eligible only when its data is current and its liquidity is sufficient; and (3) a final state is applied once, even when duplicate messages arrive.

2.5 Failure recovery workflow

Failure recovery (US-08) is the most safety-critical part of the protocol because a wrong retry could create a duplicate payment. Figure 4 shows the decision flow. The key principle is to check the true state of the remote participant before re-sending any instruction.

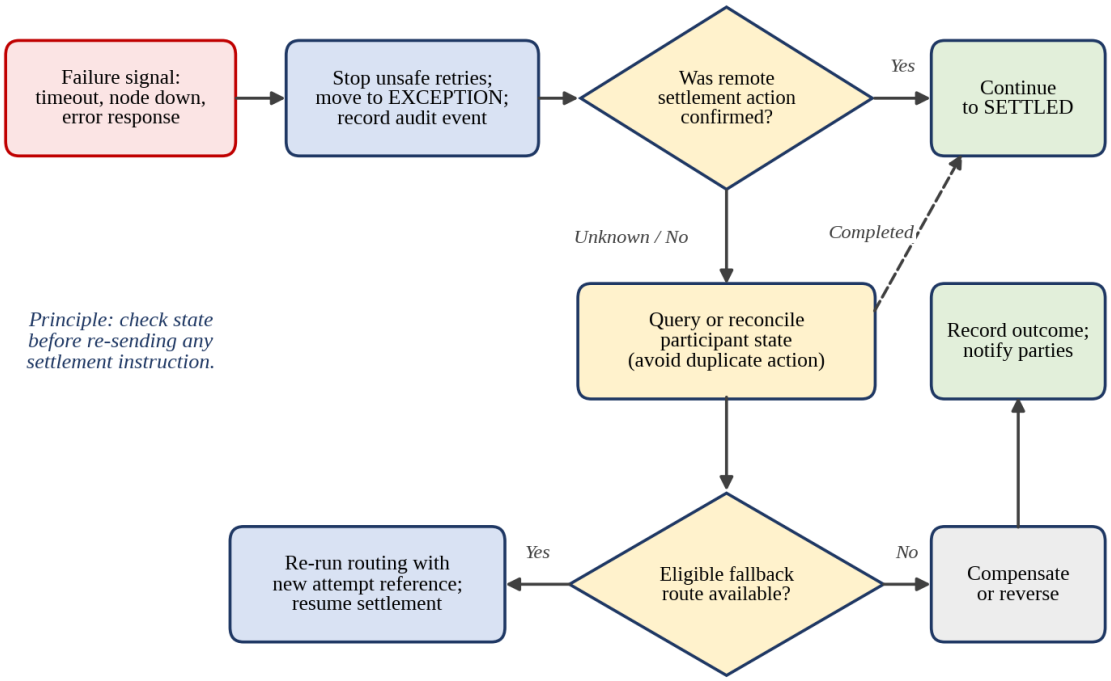


Figure 4: Failure detection, reconciliation, fallback, and compensation workflow

3. Define Acceptance Criteria

Acceptance criteria are written in Given–When–Then form. Each criterion is observable and can be converted into an automated test, which makes it usable in the Definition of Done.

US-01 – Transaction validation

- Given a transfer request is received, when required fields are missing or invalid, then the system rejects it and returns a specific validation reason.
- Given all required fields are valid, when validation completes, then the transaction is marked validated and proceeds to the compliance stage.

US-02 – Unique ID and idempotency

- Given a request with a new idempotency key, when it is accepted, then the system creates one transaction ID and stores the request result.
- Given the same request is submitted again with the same idempotency key, when it is processed, then the existing result is returned and no second payment is created.

US-03 – Sanctions and AML screening

- Given a transaction is ready for screening, when screening returns a confirmed block, then routing and settlement are stopped and the result is logged.
- Given screening returns a potential match requiring review, when the result is received, then the transaction is placed in a review state and cannot settle until cleared by an authorized process.
- Given screening returns clear, when the result is recorded, then the transaction may proceed to liquidity and route evaluation.

US-04 – Liquidity availability

- Given eligible nodes report balances, when route candidates are evaluated, then any route that cannot satisfy the required currency amount and configured liquidity buffer is excluded.

- Given a balance is stale or unavailable, when route selection runs, then the node is treated according to configured freshness policy and is not assumed to have sufficient liquidity.

US-05 – FX quote and volatility

- Given an FX quote is retrieved, when its validity period has expired, then the system does not execute conversion using that expired quote.
- Given multiple valid quotes are available, when they are compared, then the system records the rate, spread, expiry, and selected quote used in route scoring.

US-06 – Dynamic route selection

- Given several compliant routes have current data, when the routing engine scores them, then each route is evaluated using configured fee, time, liquidity, FX, and risk weights.
- Given no route meets compliance and liquidity constraints, when route selection completes, then the transaction is held or rejected with a recorded reason rather than routed through an ineligible path.
- Given a route is selected, when the decision is finalized, then the system records the chosen path and its decision inputs.

US-07 – Transaction-state consistency

- Given a transaction is pending settlement, when required participant confirmations are received, then the coordinator advances it to the defined settled state and records the confirmations.
- Given a required confirmation is missing or contradictory, when the timeout or consistency check occurs, then the transaction remains non-final and enters a controlled exception state.
- Given a transaction is already final, when a duplicate event arrives, then the system does not apply a second final state transition.

US-08 – Failure recovery

- Given a selected node becomes unavailable before final settlement, when the failure is detected, then the system stops unsafe retries and evaluates an eligible fallback route.
- Given the system cannot determine whether a remote settlement action completed, when recovery begins, then it queries or reconciles participant state before attempting a potentially duplicate action.
- Given settlement cannot safely continue, when recovery rules require compensation or reversal, then the transaction enters the appropriate recovery state and an audit event is recorded.

US-09 – Audit and monitoring

- Given a transaction changes state or route, when the event is committed, then the event records the transaction ID, timestamp, state/decision, and relevant participant response.
- Given an authorized operator searches by transaction ID, when the record exists, then the operator can view its status and permitted audit details.

US-10 – Automated reconciliation

- Given participant records for a settlement period are available, when reconciliation runs, then matching entries are marked reconciled and discrepancies are listed.
- Given a mismatch is found, when the reconciliation result is saved, then an exception is created with enough information for authorized investigation.

3.1 Measurable quality targets

The Given–When–Then criteria describe behavior. The following targets make the quality of that behavior measurable. They are proposed for the Sprint 1 test environment and are refined as real integrations are added.

Table 6: Measurable acceptance targets for Sprint 1 stories

Story	Measure	Target
US-01	Invalid requests rejected with a specific reason code	100% of the defined invalid test cases
US-01	Validation response time (95th percentile, test environment)	Under 200 ms
US-02	Duplicate requests that create a second transaction	0 in the duplicate and concurrency test suite
US-03	Block and review outcomes that reach route execution	0 in the screening test suite
US-04	Routes with stale or insufficient balances selected	0 in the liquidity test suite
US-09	State changes with a complete audit event	100% of state changes in the sandbox workflow
All	Statement coverage of new code by automated tests	At least 80%

4. Prioritize the Product Backlog

The backlog is prioritized using business value, risk reduction, dependencies, and the need to establish a safe end-to-end transaction flow. Priority labels are MoSCoW-style: **Must** = required for a viable safe flow; **Should** = important but can follow the first increment; **Could** = useful later.

Table 7: Prioritized product backlog

Priority	Story ID / Item	Business value	Dependency	Reason
P1 – Must	US-01 Validation	Prevents malformed instructions	None	Entry point for all processing
P1 – Must	US-02 ID/idempotency	Prevents duplicate payment requests	US-01	Core safety control
P1 – Must	US-03 Compliance screening	Prevents prohibited transactions	US-01, participant rules	Must precede routing/settlement
P1 – Must	US-04 Liquidity check	Avoids infeasible routes	Node balance interface	Required for route eligibility
P1 – Must	US-05 FX quote validation	Controls conversion cost and quote expiry	FX provider interface	Required for multi-currency route scoring
P1 – Must	US-06 Basic dynamic routing	Chooses an eligible efficient path	US-03, US-04, US-05	Central project capability
P1 – Must	US-07 State management	Maintains transaction status and finality	US-02, participant confirmations	Required for safe settlement
P1 – Must	US-09 Audit events	Provides traceability	Transaction state model	Supports operations and compliance
P2 – Should	US-08 Advanced fallback/recovery	Improves resilience	US-07, participant query APIs	Build after core state handling
P2 – Should	US-10 Automated reconciliation	Reduces manual effort	US-07, participant records	Depends on stable transaction events

Priority	Story ID / Item	Business value	Dependency	Reason
P3 – Could	Predictive liquidity analytics	May improve future routing	Historical quality data	Enhancement after core protocol

4.1 Weighted scoring (WSJF) cross-check

To reduce bias, the product owner and the team cross-checked the MoSCoW order with a Weighted Shortest Job First (WSJF) score: (business value + time criticality + risk reduction) divided by job size. Values use the modified Fibonacci scale agreed in the workshop; job size is the story-point estimate from Section 6. The values are workshop judgments and should be revisited as the market and regulatory context change.

Table 8: WSJF scores for the backlog

Story	Business value	Time criticality	Risk reduction	Job size	Sum	WSJF
US-01	5	8	5	3	18	6.00
US-02	8	8	8	5	24	4.80
US-03	8	8	13	5	29	5.80
US-04	8	5	8	5	21	4.20
US-05	8	5	5	5	18	3.60
US-06	13	5	5	8	23	2.88
US-07	13	5	8	8	26	3.25
US-08	8	3	13	8	24	3.00
US-09	5	5	8	3	18	6.00
US-10	5	2	5	8	12	1.50

The WSJF scores rank the small, safety-related stories first (US-01, US-09, US-03, US-02), which agrees with the Sprint 1 selection. US-06 and US-07 score lower only because they are large; they are still scheduled early because they depend on the Sprint 1 foundation and deliver the central capability. The product owner therefore applies dependencies as a constraint on top of the WSJF order. US-10 scores lowest and remains in a later sprint.

4.2 Dependency sequence

The dependency chain governs the order in which work can start: **US-01 → US-02 → US-03 → (US-04, US-05) → US-06 → US-07 → (US-08, US-10)**, with **US-09** built alongside US-07 because both rely on the same transaction event model. Sprint 1 therefore concentrates on the first part of this chain.

5. Plan the Sprint Backlog

5.1 Scrum workflow and events

The team follows Scrum with two-week sprints. Figure 5 shows the cycle from the product vision to a done increment and back to the backlog through review and retrospective feedback.

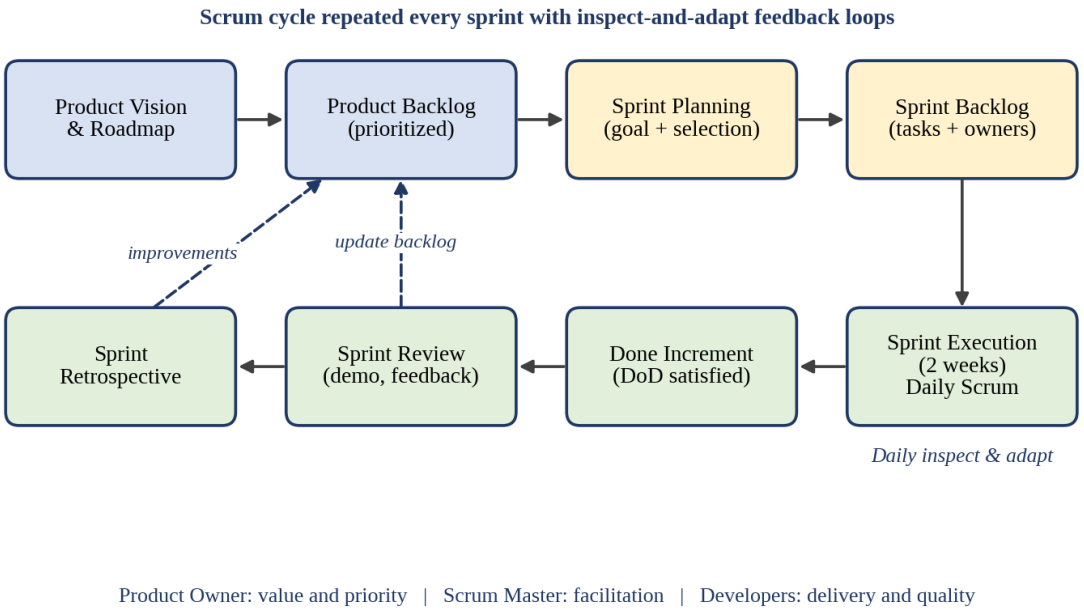


Figure 5: Scrum workflow for each two-week sprint

Table 9: Scrum events and timeboxes for a two-week sprint

Event	Timebox	Participants	Output
Backlog refinement	Ongoing (up to 10% of capacity)	Product owner, developers	Ready stories with criteria and estimates
Sprint planning	Up to 4 hours	Whole Scrum team	Sprint goal and sprint backlog
Daily Scrum	15 minutes daily	Developers (SM facilitates)	Updated plan and visible impediments
Sprint review	Up to 2 hours	Team and stakeholders	Accepted increment and updated backlog
Sprint retrospective	Up to 1.5 hours	Scrum team	Improvement actions for the next sprint

5.2 First sprint plan

Sprint duration: 2 weeks (10 working days). Team assumption: one cross-functional Scrum team with development, testing, and security support. Sprint 1 focuses on establishing a safe transaction intake and evaluation foundation using simulated or sandbox integrations. It does not claim to deliver full production-grade international settlement within one sprint.

5.3 Selected user stories

- US-01 – Validate transfer instructions.
- US-02 – Assign unique transaction IDs and handle duplicate requests idempotently.
- US-03 – Integrate a sandbox sanctions/AML screening interface and enforce clear, review, and block outcomes.
- US-04 – Implement a basic liquidity availability interface using test data.
- US-09 – Record essential transaction lifecycle and screening events.

Stretch item: if all five stories finish early, the team may start US-05 (FX quote handling). It is not part of the commitment.

5.4 Actionable sprint tasks

Table 10: Sprint 1 task breakdown

Task	Description	Story	Owner	Estimate
T1	Confirm transaction fields, states, error codes, and acceptance criteria	All	Product owner + developer	1 day
T2	Design API contracts and transaction data model	All	Developer / architect	1 day
T3	Implement request validation and error responses	US-01	Developer	2 days
T4	Implement transaction ID storage and idempotency checks	US-02	Developer	2 days
T5	Create sanctions/AML adapter using a sandbox or mock service	US-03	Developer + compliance reviewer	2 days
T6	Implement screening outcomes: clear, block, and manual review	US-03	Developer	1 day
T7	Build a mock liquidity provider and balance freshness checks	US-04	Developer	1 day
T8	Add structured audit events and transaction lookup	US-09	Developer	1 day
T9	Write unit, integration, duplicate-request, and failure-path tests	All	QA + developer	2 days
T10	Run sprint review, fix critical defects, and demonstrate the end-to-end sandbox flow	All	Whole team	1 day

5.5 Sprint timeline

Figure 6 schedules the tasks across the ten working days. T3, T4, and T5 run in parallel because they are owned by different developers and touch separate modules. Testing (T9) starts once the screening, liquidity, and audit modules are available, and the final day is reserved for the review and demonstration.

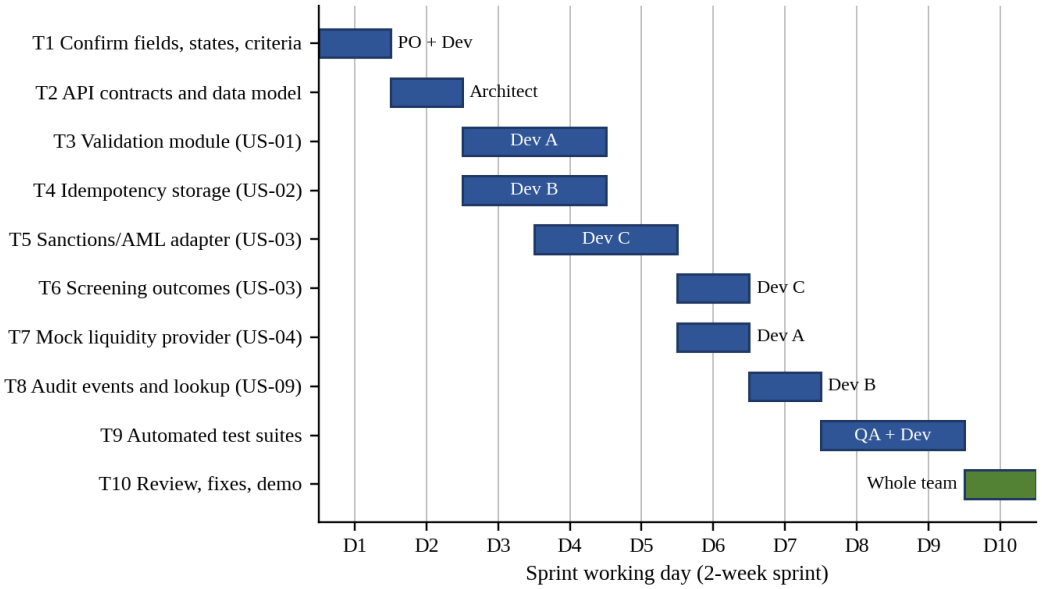


Figure 6: Sprint 1 task timeline (working days D1 to D10)

5.6 Team capacity

Capacity planning prevents over-commitment. The calculation below assumes six focused hours per person per day and subtracts the time spent in Scrum events.

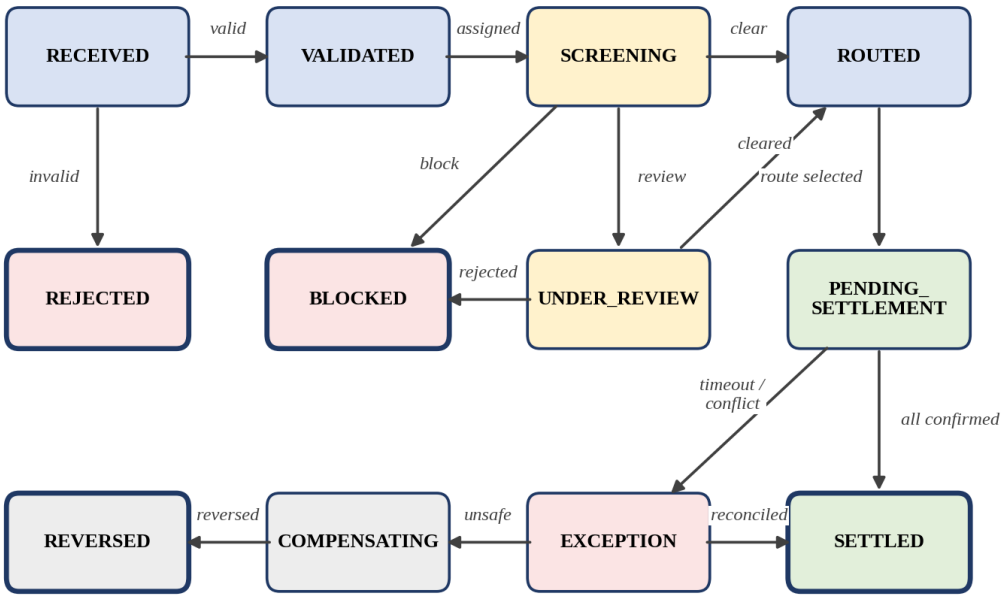
Table 11: Sprint 1 capacity estimate

Item	Calculation	Hours
Developers	3 people × 10 days × 6 hours	180
QA engineer	1 person × 10 days × 6 hours	60
Gross capacity		240
Scrum events	About 10 hours per person for 4 people	– 40
Net capacity		**200**
Planned task effort	14 person-days × 6 hours (Table 10)	84
Reserve for review, rework, environment issues, and risk	Net capacity – planned task effort	116

The reserve is deliberately large because this is the first sprint, no velocity history exists, and sandbox services may not be available on time. After the first sprint the team replaces these assumptions with measured velocity.

5.7 Initial transaction state model

T1 and T2 define the first version of the transaction state model. Figure 7 shows the complete model; Sprint 1 implements the states up to screening and review, and the later states are added with US-07 and US-08.



Double-bordered boxes are terminal (final) states; duplicate events never re-apply a final transition.

Figure 7: Transaction state model

Table 12: Transaction states and meaning

State	Meaning	Sprint
RECEIVED	Instruction accepted by the interface	1
VALIDATED	Fields and amounts passed validation; transaction ID and idempotency key assigned	1

State	Meaning	Sprint
REJECTED	Validation failed; reason returned	1
SCREENING	Sanctions/AML check in progress	1
BLOCKED / UNDER_REVIEW	Confirmed block, or potential match awaiting authorized review	1
ROUTED	Eligible route selected and liquidity reserved	2
PENDING_SETTLEMENT	Instructions sent; confirmations awaited	3
SETTLED	All required confirmations received (final)	3
EXCEPTION / COMPENSATING / REVERSED	Recovery path for failures	3–4

5.8 Definition of Ready and Definition of Done

Definition of Ready (a story may enter a sprint when):

- It follows the user-story format and is small enough for one sprint.
- Acceptance criteria are written and agreed with the product owner.
- Dependencies and test data or sandbox access are identified.
- The team has estimated it using story points.

Definition of Done:

- Acceptance criteria for the story pass.
- Code is reviewed and merged through the agreed version-control process.
- Automated unit and relevant integration tests pass.
- Security-sensitive data is not exposed in logs or error messages.
- Errors and state changes are logged with a transaction ID.
- API and configuration changes are documented.
- The increment is demonstrated in a test environment and accepted by the product owner.

6. Estimate Effort Using Story Points

6.1 Technique

The team uses Planning Poker with a modified Fibonacci scale (1, 2, 3, 5, 8, 13). Estimates represent relative effort, complexity, uncertainty, and testing effort—not hours. The values below are planning estimates and should be confirmed by the actual team. In Planning Poker each member privately selects a card, the cards are revealed together, and members with the highest and lowest values explain their reasoning before the team votes again until the estimates converge.

The baseline (reference) story is **US-01 Validation = 3 points**: its rules are known, it has no external dependency, and it needs normal unit and API tests. Other stories are compared with it. A story estimated at 13 would be split before it enters a sprint.

6.2 Story point estimates

Table 13: Story point estimates and justification

Story	Points	Relative complexity	Justification
US-01 Validation	3	Low–medium	Defined rules and field checks, with validation and error-path tests.

Story	Points	Relative complexity	Justification
US-02 ID/idempotency	5	Medium	Requires durable key storage, duplicate handling, and concurrency tests.
US-03 Screening	5	Medium–high	Requires a service adapter, policy outcomes, review/block states, and careful testing.
US-04 Liquidity check	5	Medium–high	Requires balance interface, freshness policy, and route-eligibility behavior.
US-05 FX quote handling	5	Medium–high	Requires quote data, expiry validation, and rate/spread recording.
US-06 Dynamic routing	8	High	Combines graph evaluation, multiple weighted factors, constraints, and explainable decisions.
US-07 Settlement state management	8	High	Requires state transitions, confirmations, timeouts, idempotency, and consistency tests.
US-08 Failure recovery	8	High	Requires failure detection, uncertainty handling, safe retries, and compensation logic.
US-09 Audit events	3	Low–medium	Structured event records and lookup with access control.
US-10 Reconciliation	8	High	Requires cross-node matching, mismatch detection, exception workflows, and test datasets.

The full backlog totals **58 story points** (3 + 5 + 5 + 5 + 5 + 8 + 8 + 8 + 3 + 8).

6.3 Sprint 1 commitment

Sprint 1 planning commitment: US-01 (3) + US-02 (5) + a bounded sandbox version of US-03 (5) + basic US-04 mock interface (3) + essential US-09 audit events (2) = approximately **18 story points**. The sprint should be adjusted to the team's actual historical velocity and the availability of sandbox services. The smaller sprint-scope estimates may differ from the full-story estimates above, as shown in Table 14.

Table 14: Sprint-scope estimate compared with full-story estimate

Story	Full estimate	Sprint 1 scope	Remaining for later sprint
US-01	3	3	0
US-02	5	5	0
US-03 (bounded sandbox version)	5	5	0
US-04 (mock interface)	5	3	2
US-09 (essential events)	3	2	1
Total	**21**	**18**	**3**

6.4 Release forecast

Using 18 points for the first sprint and a planning velocity of 12 to 18 points afterwards, the functional backlog of 58 points can be completed in four sprints. The forecast is revised after every sprint using the observed velocity.

Table 15: Indicative functional release forecast

Sprint	Planned content	Points
Sprint 1	US-01, US-02, US-03, US-04 (basic), US-09 (essential)	18
Sprint 2	Carry-over from Sprint 1 (3), US-05 (5), US-06 (8)	16
Sprint 3	US-07 (8), US-08 part A: failure detection and fallback (4)	12
Sprint 4	US-08 part B: reconciliation of uncertain state and compensation (4), US-10 (8)	12
Total		**58**

7. Present the Sprint Goal and Expected Deliverables

7.1 Sprint goal

Build a testable transaction-processing foundation that accepts and validates a transfer request, assigns a unique transaction ID, prevents duplicate request processing, records a sanctions/AML screening outcome, checks basic liquidity availability, and produces an auditable status.

7.2 Expected deliverables

- Documented transaction request/response API and initial transaction state model.
- Working validation module with clear error messages for invalid requests.
- Unique transaction ID and idempotency mechanism demonstrated with duplicate-request tests.
- Sandbox or mock sanctions/AML integration with clear, block, and manual-review outcomes.
- Basic liquidity interface with balance freshness and availability checks.
- Structured audit events and transaction lookup for the sandbox workflow.
- Automated unit and integration tests covering valid, invalid, duplicate, blocked, review-required, and unavailable-liquidity scenarios.
- Sprint review demonstration, updated backlog, known-issues list, and plan for the next sprint.

7.3 Success measures

- All selected story acceptance criteria pass in the test environment.
- Repeated requests with the same idempotency key do not create duplicate transactions.
- Blocked or review-required transactions cannot proceed to route execution.
- The transaction and screening outcome can be traced using the transaction ID.
- The team records actual effort and sprint completion to improve future estimation.

7.4 Sprint review demonstration plan

The Sprint Review is a working session in which stakeholders inspect the increment and give feedback. The demonstration follows the transaction workflow so that each step can be seen in order.

Table 16: Sprint review demonstration agenda

Step	Demonstration	Evidence shown
1	Submit an invalid transfer	Specific validation reason returned (US-01)
2	Submit a valid transfer, then repeat it with the same key	One transaction ID; second request returns stored result (US-02)
3	Run clear, block, and review screening cases	Correct states; blocked and review cases do not proceed (US-03)

Step	Demonstration	Evidence shown
4	Run sufficient, insufficient, and stale-balance cases	Ineligible nodes excluded (US-04)
5	Look up a transaction by ID	Status and audit trail displayed (US-09)
6	Show automated test results and known issues	Pass rate, coverage, and updated backlog

7.5 Release roadmap

Figure 8 places Sprint 1 in the longer delivery plan. The functional backlog is completed in sandbox by Sprint 4; Sprints 5 to 12 then integrate real partner environments, test performance and resilience, complete security and compliance review, and run a pilot. The diamonds mark go/no-go milestones.

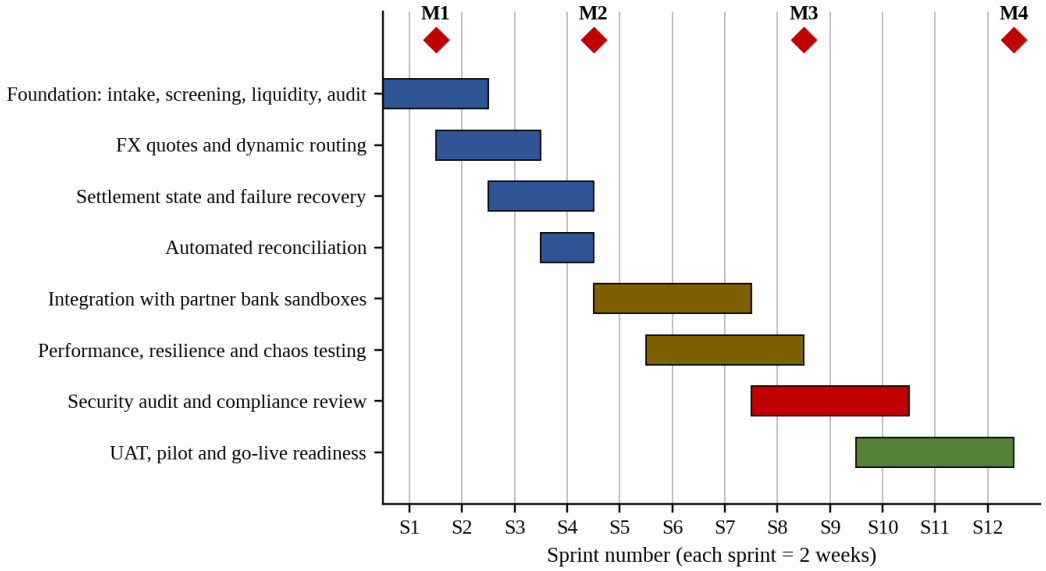


Figure 8: Indicative release roadmap over twelve sprints

Table 17: Milestones and decision gates

Milestone	End of	Exit criteria
M1 – Foundation demo	Sprint 1	Sprint goal met; velocity baseline recorded
M2 – Functional core in sandbox	Sprint 4	All ten stories pass acceptance criteria in the test environment
M3 – Integrated and tested	Sprint 8	Partner sandbox integration working; performance and resilience targets met
M4 – Pilot release	Sprint 12	Security audit and compliance review passed; UAT signed off

8. Feasibility Study

Before committing to the full programme, the project is assessed along three dimensions: whether the organization can operate and adopt the solution (**Operational Feasibility**), whether the solution can be built with available technology and skills (**Technical Feasibility**), and whether the benefits justify the cost (**Financial Feasibility**). The study uses the scope and estimates from Sections 1 to 7.

8.1 Operational Feasibility

Operational feasibility asks whether the proposed system will work in the daily operations of the participating institutions and whether people and processes are ready to use it. For a settlement protocol this includes treasury, compliance, operations, and customer-facing teams.

Process fit. The protocol automates steps that are today handled by separate systems and manual checks: validation, screening, liquidity checks, routing, and reconciliation. The workflow in Figure 3 keeps the same controls that banks already operate (validation, sanctions screening, confirmation, audit) but makes the order and the exit conditions explicit. Existing teams can therefore recognize their responsibilities in the new process, which lowers resistance.

User acceptance. Compliance specialists will accept the system only if it never bypasses screening and if review queues are manageable. Treasury managers need visible, current balances and quote information. Customers and sending banks need clear status and rejection reasons. These needs are written into US-01, US-03, US-04, and US-09, so acceptance is addressed in the backlog and is verified at each Sprint Review.

Skills and training. The delivery team needs skills in distributed systems, payments messaging, security, and testing. Operations staff need training in the monitoring console, exception handling, and the recovery procedure shown in Figure 4. A training plan is therefore delivered before the pilot (Sprint 10 to 12), supported by runbooks and sandbox exercises.

Support and operations model. Cross-border payments run across time zones, so the pilot needs a follow-the-sun or 24x7 support arrangement with defined incident severity levels, escalation paths, and service-level targets. Operations responsibilities for the live protocol are summarized in the RACI matrix below (R = responsible, A = accountable, C = consulted, I = informed).

Table 18: Operational responsibilities (RACI)

Activity	Ops	Compliance	Treasury	Recon	PO
Monitor transaction status and queues	A/R	I	I	I	I
Review screening alerts and review queue	I	A/R	I	I	C
Manage liquidity limits and buffers	C	I	A/R	I	C
Resolve settlement exceptions	A/R	C	C	R	I
Investigate reconciliation mismatches	C	C	I	A/R	I
Approve routing-weight or screening-rule changes	C	R	R	I	A

Ops = operations team; Recon = reconciliation analysts; PO = product owner.

Table 19: Operational feasibility assessment

Factor	Assessment	Mitigation	Rating
Business process fit	Controls match existing banking practice	Map each process step to a current owner during Sprint 1 refinement	High
Stakeholder acceptance	Strong need for transparency and compliance	Involve compliance and treasury in every Sprint Review	High
Team skills	Payments and distributed-systems skills must be confirmed	Architect-led design reviews; targeted training; vendor support	Medium

Factor	Assessment	Mitigation	Rating
Support model	24x7 operation is required in production	Define SLAs, on-call rotation, and runbooks before pilot	Medium
Change management	Staff move from manual to automated reconciliation	Phased rollout by corridor; parallel run before cut-over	Medium
Partner readiness	Banks must provide interfaces and test access	Interface agreements; sandbox-first integration	Medium

Conclusion: The project is operationally feasible. The main conditions are early involvement of compliance and treasury, a clear support model, and a phased rollout that includes a parallel run so that operational staff gain confidence before the manual process is retired.

8.2 Technical Feasibility

Technical feasibility asks whether the required functions can be built with proven technology, within the required performance and security limits, by a team with the necessary skills. The proposed architecture is shown in Figure 9.

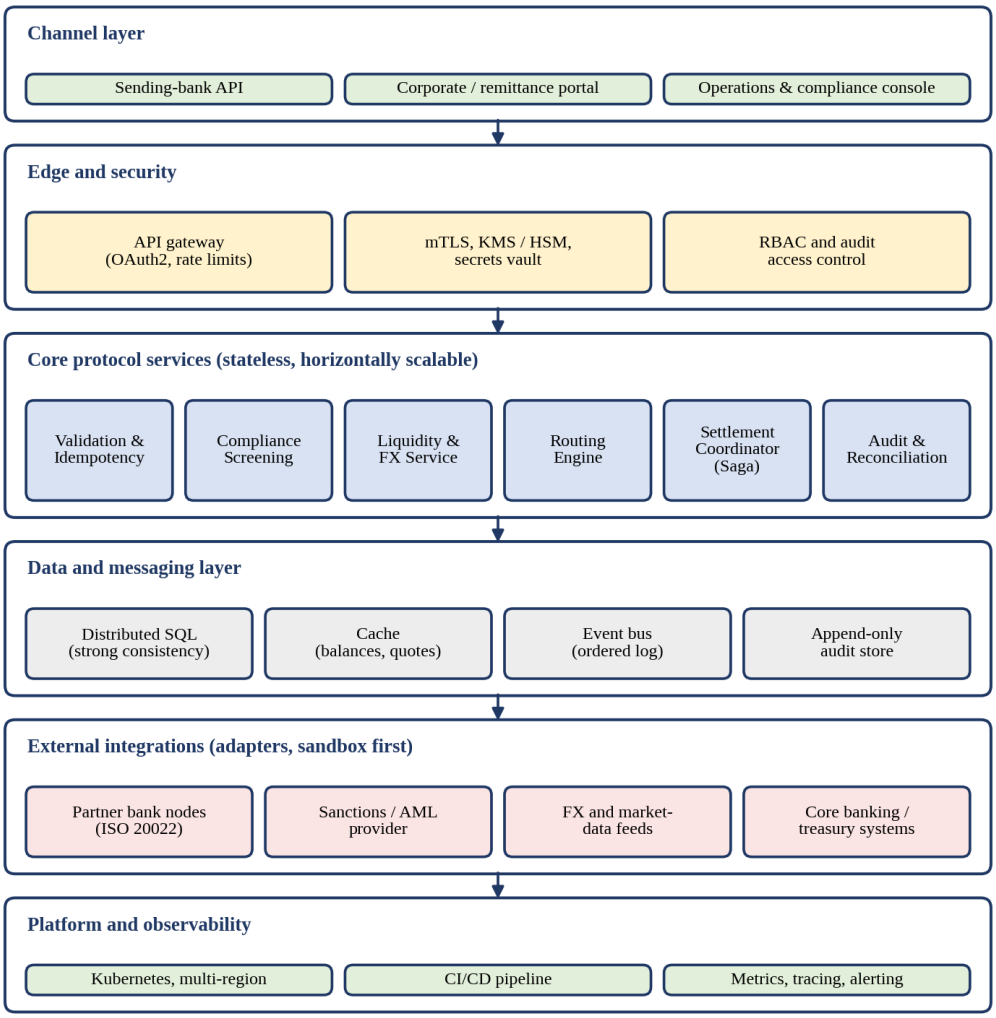


Figure 9: Proposed layered architecture of the settlement protocol

The core services are stateless and can be scaled horizontally. All durable state is held in a strongly consistent distributed database and an ordered event log, so a failed service instance can be replaced without losing a transaction. External banks, sanctions providers, and FX feeds are connected through adapters, which allows the team to use sandbox or mock implementations in the early sprints and replace them later without changing the core services.

Table 20: Proposed technology choices

Component	Proposed approach	Reason
Service implementation	Containerized microservices in a mainstream typed language (for example Java or Go)	Mature libraries, strong typing, and skills available in the market
Transaction store	Distributed SQL database with serializable transactions	Atomic state transitions and geographic distribution
Event log	Ordered, replicated event streaming platform	Reliable audit events, replay, and reconciliation input
Cache	In-memory cache for balances and quotes with expiry	Low-latency reads; freshness policy enforced by time-to-live
Routing engine	Weighted directed graph with constrained shortest-path search	Established algorithms; weights for fee, time, liquidity, FX, and risk
Settlement coordination	Saga orchestration with compensating actions	Standard pattern for multi-step distributed transactions
Messaging standard	ISO 20022 message mapping in adapters	Common payment-message standard used by banks
Security	Mutual TLS, key management service or HSM, role-based access control	Protects financial data and signing keys
Observability	Metrics, distributed tracing, and alerting	Supports operations and incident response
Deployment	Kubernetes across at least two regions with CI/CD	Availability and repeatable releases

8.2.1 Feasibility of the main requirements

Table 21: Technical feasibility by requirement

Requirement	Technical approach	Feasibility	Main risk
No duplicate or partial processing	Idempotency keys with unique constraints; single atomic state transition per event	High	Concurrency defects; mitigated by concurrency tests
Atomic multi-currency state	Saga with explicit states, timeouts, and compensation	High	Complex failure paths; mitigated by fault-injection tests
Screening before routing	Synchronous screening adapter and enforced state order	High	Provider latency; mitigated by timeouts and caching of lists
Dynamic route optimization	Graph search with weighted, constrained edges	High	Weight tuning; mitigated by explainable decision records
Real-time liquidity and FX	Event-driven balance and quote updates with freshness policy	Medium	Provider data quality and quote expiry
Synchronized records across regions	Replicated database and ordered event log	Medium	Network partitions; mitigated by reconciliation (US-10)

Requirement	Technical approach	Feasibility	Main risk
Sub-second processing	Latency budget for the internal path (Table 22)	Medium	External network time is outside system control

8.2.2 Performance feasibility

The sub-second target in the vision applies to the internal decision path, that is, from receipt of a valid instruction to a committed routing decision. It cannot cover the time that an external bank needs to confirm settlement. A latency budget shows that the internal path fits within one second with margin.

Table 22: Internal latency budget (target values)

Step	Target (ms)
Validation and request parsing	20
Idempotency check and transaction ID	10
Sanctions and AML screening call	150
Liquidity lookup (cache)	50
FX quote retrieval and validation	40
Route scoring and selection	80
State commit and audit event	100
Total internal budget	**450**

The 450 ms budget leaves about 550 ms of headroom within one second for network variation. The volume assumed in the financial model (about 50,000 transfers per month) is small compared with the capacity of the proposed technology, so the design target is a peak rate of 100 transactions per second to allow for growth and burst traffic. These figures are targets that Sprint 5 to 8 performance tests will verify.

8.2.3 Security and compliance feasibility

- Data in transit is protected with mutual TLS; data at rest is encrypted and keys are held in a key management service or hardware security module.
- Role-based access control limits who can view transaction and audit details (US-09).
- Audit events are append-only and can be hash-chained to make tampering evident.
- Logs and error messages exclude sensitive data, as required by the Definition of Done.
- Screening rules and routing weights are configuration under change control, so that compliance can review every change.

Conclusion: The project is technically feasible. Every required function maps to an established pattern. The largest uncertainties are external (partner interfaces, provider latency, and network behavior) rather than limitations of the technology, and they are addressed by sandbox-first integration, adapters, and the performance and resilience sprints.

8.3 Financial Feasibility

Financial feasibility compares the cost of building and running the protocol with the benefits it is expected to produce. All amounts are in Indian rupees (₹) and are **illustrative planning assumptions** prepared for this workshop. They must be validated with the sponsoring organization’s actual salary levels, vendor quotations, and transaction data before an investment decision.

8.3.1 Assumptions

- Initial build duration: 6 months (12 two-week sprints), matching the roadmap in Figure 8.

- Team: product owner, Scrum master, solution architect, three backend developers, two QA engineers, one DevOps and security engineer, and a compliance consultant at half time.
- Volume: 50,000 transfers per month once fully in use; benefits ramp to 60% in the first year after go-live and 100% afterwards.
- Discount rate: 12% per year; evaluation period: three years after go-live.
- Costs are in current prices; inflation is not modelled.

8.3.2 Initial build cost

Table 23: Personnel cost for the six-month build

Role	Number	Monthly cost each	Six-month cost
Product Owner	1	₹1,80,000	₹10,80,000
Scrum Master	1	₹1,50,000	₹9,00,000
Solution Architect / Tech Lead	1	₹2,40,000	₹14,40,000
Backend Developers	3	₹1,60,000	₹28,80,000
QA / Test Engineers	2	₹1,20,000	₹14,40,000
DevOps & Security Engineer	1	₹1,80,000	₹10,80,000
Compliance Consultant (0.5 FTE)	0.5	₹2,00,000	₹6,00,000
Total personnel			**₹94,20,000**

Table 24: Total initial build cost

Cost item	Amount
Personnel (Table 23)	₹94,20,000
Cloud infrastructure (dev, test, sandbox, 2 regions)	₹9,00,000
Third-party sandboxes and data feeds (sanctions, FX, messaging)	₹8,00,000
Security testing and compliance audit	₹6,00,000
Tools and licences (CI/CD, tracking, monitoring)	₹2,50,000
Subtotal	**₹1,19,70,000**
Contingency (10% of subtotal)	₹11,97,000
Total initial investment	**₹1,31,67,000** (about ₹1.32 crore)

8.3.3 Operating cost and benefits

Table 25: Annual operating cost after go-live

Item	Annual cost
Production cloud and network (multi-region)	₹48,00,000
Support and operations team (4 members)	₹72,00,000
Licences, data feeds and compliance subscriptions	₹20,00,000
Total operating cost per year	**₹1,40,00,000**

Table 26: Expected annual benefits at full run-rate

Benefit	Annual value
Fee and FX-spread savings (₹40 x 600,000 transfers)	₹2,40,00,000
Reduced manual reconciliation (3 FTE x ₹90,000 x 12)	₹32,40,000

Benefit	Annual value
Fewer delayed or failed settlements (penalties, rework)	₹15,00,000
Total annual benefit	**₹2,87,40,000**

8.3.4 Cash flow and investment measures

Table 27: Three-year cash flow

Item	Year 0 (build)	Year 1	Year 2	Year 3
Benefits	₹0	₹1,72,44,000	₹2,87,40,000	₹2,87,40,000
Build / operating cost	– ₹1,31,67,000	– ₹1,40,00,000	– ₹1,40,00,000	– ₹1,40,00,000
Net cash flow	₹-1,31,67,000	₹32,44,000	₹1,47,40,000	₹1,47,40,000
Cumulative	₹-1,31,67,000	₹-99,23,000	₹48,17,000	₹1,95,57,000

Table 28: Investment measures (base case)

Measure	Value
Total initial investment	₹1,31,67,000
Net present value at 12% (3 years)	₹1,19,71,707 (about ₹1.20 crore)
Benefit–cost ratio (present values)	1.26
Return on investment over 3 years	148.5%
Payback period after go-live	20.1 months

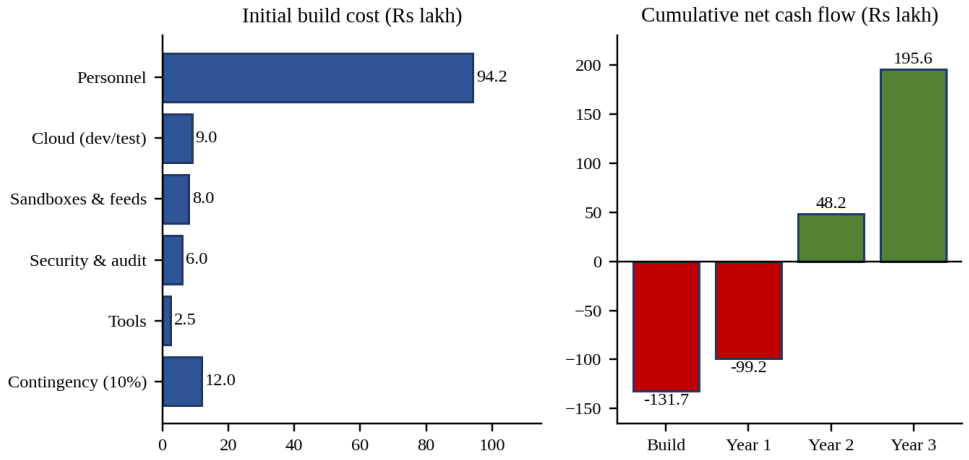


Figure 10: Initial build cost breakdown and cumulative net cash flow

8.3.5 Sensitivity analysis

Because the inputs are estimates, the investment is tested against less favorable outcomes. The table shows net present value and payback when benefits are lower than planned, when the build costs more than planned, and when both happen together.

Table 29: Sensitivity of the financial result

Scenario	NPV at 12%	Payback after go-live
Base case	₹1,19,71,707	20.1 months
Benefits 20% lower	₹2,18,838	29.8 months
Build cost 20% higher	₹93,38,307	22.2 months

Scenario	NPV at 12%	Payback after go-live
Benefits 20% lower and build cost 20% higher	– ₹24,14,562	33.4 months

The project remains attractive in the base case and when only one assumption turns out worse. If benefits fall by 20% and build costs rise by 20% at the same time, the net present value becomes negative, which shows that benefit realization and cost control need active management. The phased roadmap supports this because investment is released at the decision gates M1 to M4.

Conclusion: The project is financially feasible under the stated assumptions. Staged funding limits the exposure: the Sprint 1 cost is a small fraction of the total, and later funding depends on the results of the milestones.

8.4 Overall feasibility summary

Table 30: Feasibility scorecard

Dimension	Key finding	Rating	Condition
Operational	Fits existing controls; needs training, support model, and phased rollout	Feasible	Parallel run before cut-over
Technical	Proven patterns; external interfaces are the main uncertainty	Feasible	Sandbox-first integration; performance tests
Financial	Positive NPV and payback of about 20 months after go-live	Feasible	Validate cost and volume assumptions

Recommendation: proceed with Sprint 1 as a low-cost proof of the transaction foundation, and review operational, technical, and financial assumptions at each decision gate before releasing further funding.

9. Risk Register

Risks are scored by probability (P) and impact (I) on a three-level scale (L = low, M = medium, H = high). The Scrum master maintains the register and reviews it in each sprint retrospective.

Table 31: Project risk register

ID	Risk	P	I	Mitigation
R1	Sandbox or partner interfaces are unavailable on time	M	H	Mock adapters in Sprint 1; early interface agreements; keep adapters replaceable
R2	Duplicate or partial settlement due to concurrency or retry defects	L	H	Idempotency keys, atomic state transitions, concurrency and fault-injection tests
R3	Regulatory rules change or are interpreted differently by regions	M	H	Configurable screening rules; compliance consultant in refinement; change control
R4	Screening produces too many alerts for manual review	M	M	Tune thresholds with compliance; track alert volume as a metric
R5	Stale or incorrect liquidity or FX data causes poor route choices	M	H	Freshness policy; exclude stale nodes; record decision inputs
R6	Performance target is not met under load	M	M	Latency budget; early load tests; caching; horizontal scaling
R7	Estimates are inaccurate because the team has no velocity history	H	M	Conservative first-sprint commitment; re-estimate after Sprint 1

ID	Risk	P	I	Mitigation
R8	Key team members are unavailable or leave	L	M	Pair programming; shared code ownership; documentation
R9	Benefits are lower than planned	M	M	Track benefit metrics; staged funding at decision gates
R10	Security breach or data exposure	L	H	Encryption, access control, security testing, and log-hygiene checks

10. Conclusion and References

Conclusion

This Agile sprint plan converts the Cross-Border Settlement & Liquidity Routing Protocol problem into prioritized epics, testable user stories, measurable acceptance criteria, and a realistic first sprint. The initial sprint establishes safe transaction intake, idempotency, screening, basic liquidity visibility, and auditability. Later sprints can add FX-aware dynamic routing, distributed settlement coordination, advanced recovery, and automated reconciliation. Scrum supports incremental feedback and reprioritization, while explicit acceptance criteria and a Definition of Done help ensure that financial safety, compliance, reliability, and traceability are built into the development process.

The feasibility study adds confidence that the plan can be executed: the solution fits the operating environment of the participating institutions, relies on proven technical patterns, and offers a positive return under reasonable assumptions, with staged decision gates to control risk.

References

1. Schwaber, K. and Sutherland, J. (2020). The Scrum Guide.
2. Cohn, M. (2004). User Stories Applied: For Agile Software Development. Addison-Wesley.
3. Cohn, M. (2005). Agile Estimating and Planning. Prentice Hall.
4. Garcia-Molina, H. and Salem, K. (1987). Sagas. ACM SIGMOD Record, 16(3).
5. International Organization for Standardization. ISO 20022: Financial services – Universal financial industry message scheme.
6. Financial Action Task Force. The FATF Recommendations on anti-money laundering and countering the financing of terrorism.